

Hybrid Stochastic Number and Its Neural Network Computation

Hongge Li^{ID}, *Member, IEEE*, and Yuhao Chen^{ID}, *Student Member, IEEE*

Abstract—Stochastic computing (SC) is unique in that it is a type of arithmetic computation based on stochastic numbers (bitstream) instead of binary numbers (BNs). Stochastic number (SN) represents and carries information in the form of pseudo-analog probabilities by CMOS gate circuits. The renewed success of the stochastic number system is mainly related to super low power consumption and high reliability for edge computing. In fact, the stochastic number is a nonpositional number representation that is intrinsically sequential and consequently used for certain important arithmetic operations (such as addition/subtraction and multiplication), and corresponds to a super low area circuit. This article proposes a novel hybrid number system of BNs and stochastic number representation, called hybrid stochastic number (HSN). This study introduces the basic theoretical aspects of the HSN and demonstrates the properties of hybrid stochastic computing (HSC). The hardware implementation of deep neural network with HSC is fabricated using a standard 40-nm low-power CMOS process, with a core area of 0.53 mm², power of 102.3 mW, and clock of 400 MHz, which has 4544 multiply accumulation operations (MACs).

Index Terms—Application-specific integrated circuit (ASIC), deep neural network, hybrid stochastic computing (HSC), hybrid stochastic number (HSN), stochastic number.

I. INTRODUCTION

THE well-known redundant number representations make it feasible to perform addition with carry-free propagation chains to speed up arithmetic operations [1], [2]. Currently, hardware computing is constrained by stringent application requirements, such as extremely low cost, low power, and high performance. Stochastic computing (SC) was proposed in the 1960s as a low-cost, low-power alternative to conventional binary number (BN) computing [3], [4]. SC is a computing problem based on the probability and means of the bitstream, and stochastics possessing special properties can be easily generated using simple gate circuits replacing complex circuits of BNs [5], [6].

The speed advantage of BNs for digital computations is significant. However, its arithmetic structure is characterized by the need for a large communication width between modular

processors to realize high-performance computing systems. In the SC paradigm, the computation is performed on a bitstream instead of a BN. The stochastic bitstream is represented by the probability of firing “1” based on the stream length of the accumulation [3], [4], [5], [6], [7]. The required computing power of the SC was completely distributed throughout the low-cost gate circuit. In other words, classic SC with high fault tolerance saves considerable hardware and power resources at the expense of efficiency [3], [7], [8]. Despite this advantage and its inherent properties, SC cannot replace BN due to their high computational latency and relatively low accuracy. The high latency is essentially caused by the low information-carrying ability of 1-bit stochastic stream in conventional SC. When the bitstream length of the stochastic logic is equal to L , it can only represent L different data points. Clearly, the efficiency of carrying information for a bitstream is weaker than that for a BN.

The digital computation of SC is performed on a stochastic bitstream instead of a BN. To accommodate negative numbers, many SC systems employ the bipolar format, where the SC range effectively becomes $[-1, 1]$. Gaines [8] presented dual-rail unipolar and bipolar number formats, along with the basic circuits for each format. The stochastic number formats have been discussed in some papers, which has a significant advantage in the utilization of hardware for processing complex functions [9], [10], [11]. Alireza et al. [12] proposed an approximate hybrid binary–unary computing for univariate math function computation. Some researchers have attempted to improve the efficiency of SC encoding, using different technologies to address latency [13], [14], [15]. Some researchers have improved the computational accuracy of SC with different technologies, such as deterministic and so on [16], [17]. Xia et al. [18] proposed the neural synaptic plasticity-inspired computing and reconfigurable neural synaptic plasticity for implementing high computing efficiency in convolutional neural network (CNN) accelerator. In general, SC has been attempted to focus on a range of specialized applications, such as neural networks [21], [22], [23], polynomial calculations [19], [20], and image processing [12], [24].

However, Poppelbaum [25] has observed that “short sequences are untrustworthy” and that a major drawback of SC is low bandwidth and therefore low computational speed. There are certain important theoretical discoveries related to the fundamental concept of stochastic number (SN) that have attracted little attention, though they have positive implications for SC. Parhi [26] has presented general methods to analyze

Manuscript received 6 July 2023; revised 26 September 2023 and 20 October 2023; accepted 4 November 2023. Date of publication 27 November 2023; date of current version 27 February 2024. This work was supported in part by the National Natural Science Foundation of China under Grant No. 62071019 and Grant No. U22A6002. (Corresponding author: Hongge Li.)

The authors are with the College of Electronic Information Engineering, Beihang University, Beijing 100191, China (e-mail: honggeli@buaa.edu.cn; zy1902608@buaa.edu.cn).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TVLSI.2023.3332170>.

Digital Object Identifier 10.1109/TVLSI.2023.3332170

1063-8210 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See <https://www.ieee.org/publications/rights/index.html> for more information.

stochastic combinational logic circuits in unipolar, bipolar, and hybrid encoding formats. Methods that use bipolar and hybrid formats can realize a simpler combinational logic structure for the synthesis of any polynomial. Canals et al. [27] proposed a new encoding method that uses a fractional form and extends the range of bitstream representations to the entire real-number axis.

To solve the intrinsic fundamental problem and major design challenges of classical SC, we propose the HSN, and hybrid stochastic computing (HSC) method and discuss the properties of the HSN representation. The main contributions of this study are as follows.

- 1) Hybrid stochastic number (HSN) representation method from the BN and stochastic number is first propounded in the number system of BN or SC.
- 2) Unify the representation of BN, SN, and HSN. The mathematical description and conversion relationship between the BN, SN, and HSN are discussed.
- 3) HSN realizes high efficiency and low latency compared to classical SN, because of not using a converter between the BN and SN.

II. NUMBER SYSTEM

A. Classical Binary Number

Number-representation methods have advanced with the evolution of science. The positional number system is the most conventional fixed-radix representation method, where the unit corresponding to each position is a constant multiple of the unit for its right neighboring position. The BN, whose digit set is $[0, 1]$ is also a positional number system and is nonredundant. It is easy to implement in CMOS logic circuits and is widely used in central processing unit (CPU), graphics processing unit (GPU), and neural processing unit (NPU).

In the early 1960s, Avizienis [1] defined a class of signed-digit (redundant) number systems with symmetric digit sets $[-\alpha, \alpha]$ and radix $r > 2$, where α is an arbitrary integer. Subsequently, the redundant signed-digit number systems with general, possibly asymmetric, the digit sets of the form $[-\alpha, \beta]$ were studied as tools for unifying all redundant number representations used in practice. Extended signed-digit number representation systems were defined by Parhami [2], Gaines [8], and Alaghi et al. [9], and novel HSN systems are expanded as shown in Fig. 1.

B. Conventional Stochastic Number

A conventional SN uses a single bitstream to represent data, and the expectation of the stochastic bitstream ranges from 0 to 1 (unipolar) or from -1 to 1 (bipolar). Here, we mainly focus on unipolar SN. The random number generator (RNG) in the sequence is compared with the BN, and 1 bit of the encoding output is generated as shown in Fig. 2. If BN is larger than a random number, the output of the comparator is high-level “1”; otherwise, the output is low level “0.” The expectation of the output bitstream is equal to the probability that the BN is larger than a random number. When an SN is in a unipolar form, the probability of an SN of bitstream length

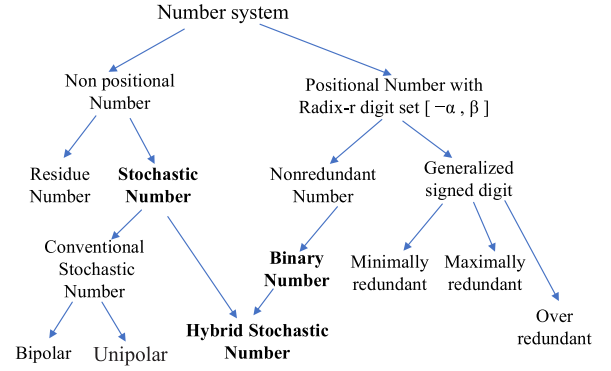


Fig. 1. SN, HSN, and classic BN system.

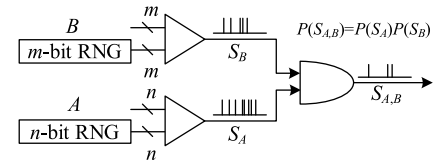


Fig. 2. SC with unipolar stochastic bitstreams.

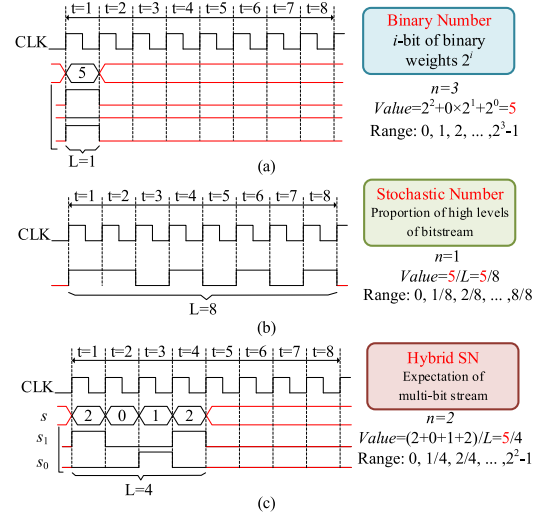


Fig. 3. Different representations of (a) BN, (b) SN, and (c) HSN.

L , is defined as the relative frequency of the encoding of 1

$$\hat{p} = \hat{p}(\text{SN} = 1) = \frac{\sum_{j=0}^{L-1} \text{SN}(j)}{L} \in [0, 1] \quad (1)$$

where $\sum_{j=0}^{L-1} \text{SN}(j)$ is the number of 1's in the sequence compared, and $\hat{p}$ is the estimated value of the probability. The SN representation of a particular stream is not defined by the individual value of a bit in a specific position; thus, the SC belongs to a nonpositional number representation. SN uses a redundant number system, in which there are redundant representations for each number, $\hat{p}(\text{SN} = 1)$. In redundant representation, one algebraic value may be represented in more than one way. For example, with bitstream length $L = 3$, there are three ways to represent $1/3$: $(0, 0, 1)$, $(0, 1, 0)$, and $(1, 0, 0)$.

The stochastic number associated with mapping from space-to-time encoding leads to SC arithmetic, which can be

implemented using simple logic gate circuits. As another nonpositional representation, SC improves the robustness of circuits subject to random faults or component variability.

C. Hybrid Number of Stochastic and Binary

A conventional fixed-radix BN system is typically based on a positive integer radix (base) $r = 2$ and an implicit digit set $[0, 1]$. This is referred to as a positional-weighted (radix) system. Hence,

$$B = b_{n-1}r^{n-1} + \dots + b_1r^1 + b_0r^0 = \sum_{i=0}^{n-1} b_i \cdot r^i. \quad (2)$$

Definition 1: Considering the representation of the SN in (1) with respect to the BN B in (2), we have

$$\begin{aligned} \hat{E}[S] &= \sum_{i=0}^{n-1} r^i \cdot \hat{p}(S_i = 1) = \sum_{i=0}^{n-1} r^i \left(\frac{\sum_{j=0}^{L-1} S_i(j)}{L} \right) \\ &= \frac{1}{L} \left(\sum_{j=0}^{L-1} \sum_{i=0}^{n-1} S_i(j) \cdot 2^i \right). \end{aligned} \quad (3)$$

Here, S_i represents the expanded SN representation, which is a 2-D matrix consisting of 0–1 number stream. Every row of S_i forms an SN, S_i represents the i th row of HSN. And, $S_i(j)$ represents the bit at i th row and j th column of HSN. $\hat{E}[S]$ is the estimated value of the expectation of HSN. Based on the representation system of (3), it shows that the HSN consisting of multiple SN with different weights is actually a binary sequence. This expanded SN representation system is called an HSN, which still uses expectations to represent data and probabilistic theory for computing [28], [29]. However, each bitstream of the HSN has a positional weight value based on BN, where the weight is 2^i . S represents an n -bit HSN as

$$S = S_{n-1}2^{n-1} + \dots + S_12^1 + S_02^0 \quad (4)$$

where $S_{n-1}, \dots, S_1$ and S_0 represent different SNs based on the positional weight of the HSN. The expectation of n -bit HSN is expressed as follows:

$$E[S] = 2^{n-1}E[S_{n-1}] + \dots + 2^1E[S_1] + E[S_0] \quad (5)$$

where $E[S]$ represents the expectation of different S_i of HSN. The range of values represented by n -bit HSN is larger than that of a single SN, which the number ranges is from 0 to $2^n - 1$.

Example 1: We constructed an example of a binary sequence with radix-2 from two elementary binary sequences with a bitstream length L of 16. Select elementary binary sequences with probabilities of b_1 and b_0 that

$$\begin{aligned} S &= \begin{bmatrix} S_1 \\ S_0 \end{bmatrix} \\ &= \begin{bmatrix} 1101 & 1111 & 1101 & 0011 \\ 1011 & 0100 & 1001 & 1111 \end{bmatrix}, \quad \hat{S}_1 = b_1 = 12/16 \\ &\quad \hat{S}_0 = b_0 = 10/16. \end{aligned}$$

Subsequently, a new hybrid stream (HS) sequence HSN is constructed from these two stream sequences, as follows:

$$S = S_1 \times 2^1 + S_0 \times 2^0 = 3213 \ 2322 \ 3203 \ 1133$$

$$\begin{aligned} \hat{E}[S] &= \frac{\sum_{j=0}^{L-1} \left[\sum_{i=0}^{n-1} S_i(j) \cdot 2^i \right]}{L} = b_1 \times 2^1 + b_0 \times 2^0 \\ &= \frac{3 + 2 + 1 + 3 + \dots + 1 + 1 + 3 + 3}{16} \\ &= \frac{34}{16} = 2 \times \frac{12}{16} + \frac{10}{16}. \end{aligned}$$

Hence, based on the definition of HSN, we can construct HSN sequences of arbitrary finite lengths from a set of SN sequences with binary weight.

Definition 2: Observing (3) of HSN.

1) It is BN, $S_i(j) = b_i$ at each j , b_i is 0 or 1

$$\hat{E}[S] = \frac{\sum_{j=0}^{L-1} \left[\sum_{i=0}^{n-1} S_i(j) \cdot 2^i \right]}{L} = \sum_{i=0}^{n-1} b_i \cdot 2^i, \text{ iff, } n \geq 1 \text{ and } L = 1.$$

2) It is conventional SN

$$\hat{E}[S] = \frac{\sum_{j=0}^{L-1} \left[\sum_{i=0}^{n-1} S_i(j) \cdot 2^i \right]}{L} = \frac{\sum_{j=0}^{L-1} S(j)}{L}, \text{ iff, } n = 1, \text{ and } L > 1.$$

3) It is HSN

$$\hat{E}[S] = \frac{\sum_{j=0}^{L-1} \left[\sum_{i=0}^{n-1} S_i(j) \cdot 2^i \right]}{L}, \text{ iff, } n \geq 1 \text{ and } L \geq 1.$$

The representations of SN and BN are two extremes of HSN and are two special types of HSN as shown in Fig. 3. The HSN is represented by a parallel multi-bit stream and follows the position number rule of BNs. The amount of information carried by a multi-bit stream HSN or the numerical representation accuracy, has been significantly improved. Moreover, the bitstream length (L) of HSN is significantly shortened for the same precision number representation, which effectively addresses the problem of high latency in the traditional SN representation.

Unlike conventional BN systems, HSN systems are redundant number representation. Encoding the same number with an HSN may yield multiple representations, similar to an SN. For example, $(2, 0, 1, 2)_{\text{HSN}}$, $(2, 1, 1, 1)_{\text{HSN}}$, and $(1, 3, 1, 0)_{\text{HSN}}$ have the same value because the mean values of the three HSN are all equal to $5/4$. Therefore, HSN systems are representative of systems with redundant numbers.

Example 2: Binary number x is encoded as an HSN. The first step of generation is splitting x into n BN with a smaller bit width, $\text{BN}_0, \text{BN}_1, \dots, \text{BN}_{n-1}$. The relationship between x and BN_i is described as follows:

$$x = \text{BN}_{n-1}2^{n-1} + \dots + \text{BN}_12^1 + \text{BN}_02^0. \quad (6)$$

The bit width of BN_i is k_i , $k_i \in N^+$. The second step is to encode $\text{BN}_0, \dots, \text{BN}_{n-1}$ as an SN. They were compared with $k_{\max}$ -bit RNG. Thus, the expectation of n SNs carries the same coefficient of $2^{-k_{\max}}$. $k_{\max}$ is the maximum value of k_i

$$\begin{cases} \text{if } \text{BN}_i > \text{RNG}(j), & S_i(j) = 1 \\ \text{else} & S_i(j) = 0 \\ E[S_i] = \text{BN}_i / 2^{k_{\max}} & (i = 0, 1, \dots, n-1). \end{cases} \quad (7)$$

Definition 3: By combining (4)–(7), the expectation of the HSN can be obtained as follows:

$$E[S] = \sum_{i=0}^{n-1} 2^i E[S_i] = \frac{\sum_{i=0}^{n-1} 2^i \text{BN}_i}{2^{k_{\max}}} = \frac{x}{2^{k_{\max}}}. \quad (8)$$

The HSN output successfully represents x , and its expectation has a coefficient of $2^{-k_{\max}}$. An example of the generation

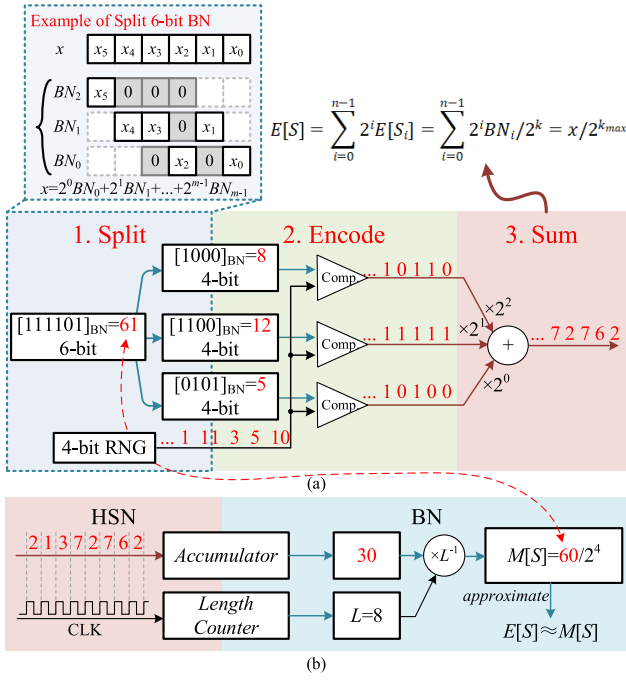


Fig. 4. Encoding and decoding processes of the HSN. (a) 6-bit BN is encoded as 3-bit HSN. (b) HSN is decoded as BN.

of a 3-bit HSN is provided in Fig. 4(a) to explain the details of the encoding circuit.

As aforementioned, the value represented by the HSN is determined by its expectation, which is obtained by counting the mean of an HSN with infinite length, which is impossible in real circuits. Therefore, the mean of an HSN with a finite length stream is counted to obtain the approximate expectation, as shown in Fig. 4(b)

$$E[S] = \lim_{L \rightarrow \infty} M[S] = \lim_{L \rightarrow \infty} \sum_{t=1}^L S(t)/L \quad (9)$$

where $M[S]$ represents the mean value of S , and S is an HSN. For the easy distinction between rows and columns in the HSN, $S(t)$ represents the number consisting of the t th column in S . This operation can be realized using an accumulator and counter. An accumulator is used to count $\sum_{t=1}^L S(t)$, and a counter is used to count bitstream length L . When $L = 2^n, n \in \mathbb{N}$, the division can be replaced with a shifting.

D. Arithmetic Operation of HSC

1) *Multiplication*: The most common situation is the multiplication of two HSNs, as discussed in [29]. S_a and S_b represent the two HSNs, and the bitstream lengths were both L . The S_c expectation equals the product of S_a expectation and S_b expectation in the HSC. The relationship between the expectations of the inputs and outputs of the multiplication operation is expressed as (10). Multiplication operation is realized when S_a and S_b are independent

$$\begin{aligned} E[S_c] &= E[S_a \cdot S_b] \\ &= E[S_a] \cdot E[S_b] \text{ iff } S_a, S_b \text{ independent.} \end{aligned} \quad (10)$$

This type of multiplication can produce errors, the main reasons for which are twofold. First, it is difficult to achieve a completely independent relationship between the inputs. Second, the mean of the HSN with a finite length is used to approximate the expectation of the HSN; which may produce errors. An example of this method is presented in Fig. 5(a). In this example, the mean of the output HSN has no errors. However, this situation is a special case, and the error is not always 0.

Suppose that the S_a and S_b are the n -bit and m -bit HSN, respectively. By changing (10) to a positionally weighted format of the SN, S_c expression of the HSN is obtained

$$\begin{aligned} E[S_c] &= E[S_a \cdot S_b] \\ &= E \left[\left(\sum_{i=0}^{n-1} 2^i S_{a,i} \right) \left(\sum_{j=0}^{m-1} 2^j S_{b,j} \right) \right] \\ &= \sum_{i=0}^{n-1} \sum_{j=0}^{m-1} 2^{i+j} E[S_{a,i} \cdot S_{b,j}] \end{aligned} \quad (11.a)$$

$$\begin{aligned} E[S_c] &= \sum_{i=0}^{n-1} \sum_{j=0}^{m-1} 2^{i+j} E[S_{a,i}] \cdot E[S_{b,j}] \\ &= E[S_a] \cdot E[S_b] \text{ iff } S_{a,i}, S_{b,j} \text{ independent.} \end{aligned} \quad (11.b)$$

$S_{a,i}$ represents the i th SN with a weight of 2^i in S_a , $S_{b,j}$ is similar. From the expectation expressions of (11), only when two SNs are in different HSNs, they are multiplied; thus, they are required to be independent. The independence requirement can be solved using existing schemes, such as Sobol [12] or independent RNGs, and so on. As for the two SNs in the same HSN, there is no multiplication between them, so they have no independence requirement. Thus, the BN-to-SN encoding in the same HSN is allowed to use the same RNG, which does not violate the independence requirement of the multiplication operation.

As a concept of HSN in (3), the BN can be seen as a constant HSN. Based on the properties of expectation, when one of the inputs is replaced by a constant HSC, $E[S_a \cdot S_b] = E[S_a] \cdot E[S_b]$ is established unconditionally. Therefore, it is not necessary for the independence requirement of multiplication, and the operation accuracy will not be impacted by nonindependent inputs. An example of the multiplication of m -bit BN, x , and n -bit HSN, S_a , is shown in Fig. 5(b). The output is an $(m+n)$ -bit HSN, and its bit width is determined by the bit widths of the input BN and HSN. The expectation and mean of the output HSN, which realizes multiplication without any other requirements, are as follows:

$$M[S_c] = \sum_{t=1}^L x \cdot S_a(t)/L = x \cdot M[S_a] \quad (12)$$

$$E[S_c] = E[x \cdot S_a] = x \cdot E[S_a]. \quad (13)$$

In the HSC domain, nonindependent inputs be not influence the computing. The computing error is only caused by the encoding accuracy of HSN; that is, the difference between the mean of S_a and its expectation. When stream L increases to 2^{n+m-1} for n -bit BN to m -bit HSN, the encoding error is 0.

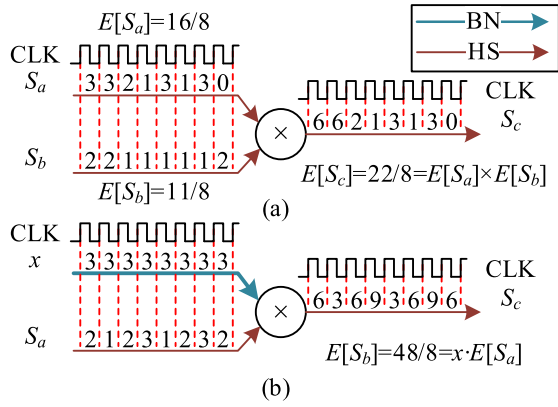


Fig. 5. Multiplication methods for HSC with different kinds of input. (a) HSN $\times$ HSN and (b) BN $\times$ SN, the bit width of output is changeable.

Therefore, its mean value is exactly equal to the expectation, and the computing of HSN $\times$ BN is completely accurate. Low-discrepancy sequences can be used as a random source in HSC, which yields a benefit on the encoding accuracy. Of course, a common LFSR or a counter in a classical SC scheme can also work in HSC. Since the accuracy influence of random sources on the whole multiplication only lies in the step of BN-to-HSN encoding, when L is small, low-discrepancy sequences have better accuracy than others. As well known, deterministic systems can achieve completely accurate computation. The computation operation of HSC is too, that is no error in the process of converting BN to HSN. Therefore, HSC can achieve completely accurate computation as the deterministic systems. Moreover, HSC implement the higher efficiency by multibit parallelism encoding, it has significant advantages in computation latency.

2) *Addition and Subtraction*: HSC adder in arithmetic operation is realized by the scaled addition, which consists of n MUXs with n -bit HSN. Or it can use a binary adder to realize addition as shown in Fig. 6(b) and (c). Similar to multiplication for HSC, the input of three types of formats is used and the output is HSN.

The common situation is that the inputs are two HSNs. S_a and S_b are two inputs of addition, and the stream lengths are both L . S_c is the expectation of output HSC equal to the sum of the expectations of input S_a and S_b . This is achieved by adding the corresponding numbers in S_a and S_b together [29]. The subtraction operation is the same as in addition, and the two operations are described in (14). The expectations and the mean values of outputs are shown in the following equation:

$$E[S_c] = E[S_a \pm S_b] = E[S_a] \pm E[S_b] \quad (14)$$

$$M[S_c] = \frac{\sum_{t=1}^L [S_a(t) \pm S_b(t)]}{L} = M[S_a] \pm M[S_b]. \quad (15)$$

Here, the stream length t takes every integer from 1 to L to generate $S_c(1), S_c(2), \dots, S_c(L)$. In one signed HSN, the situation including the negative $S(t_i)$ and the positive $S(t_j)$ is allowed and $t_i \neq t_j$.

III. NEURAL NETWORK IMPLEMENTATION OF HSC

A. Activation Function Approximation

The calculation circuit for the activation function was designed in the form of successive approximations. Function

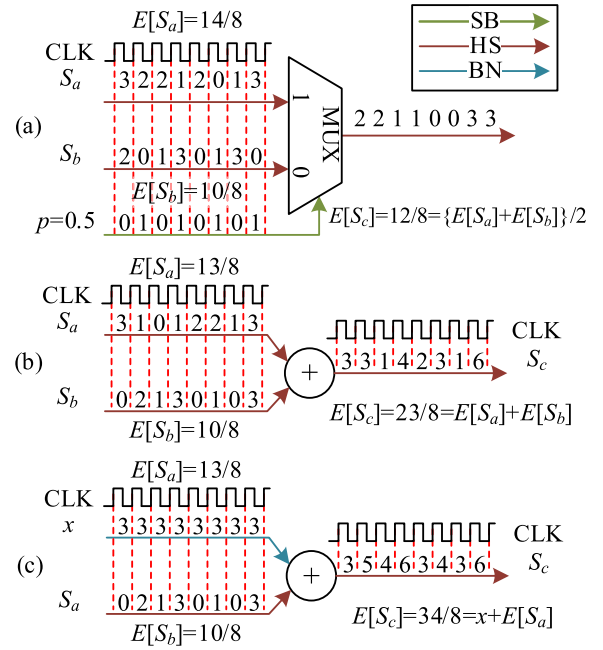


Fig. 6. Addition allows BN, SN, or HSN as input. (a) Scaled addition using MUX (Multiplexer), (b) HSC adder with two inputs of HSN, and (c) HSC adder with two inputs of BN and HSN.

calculation in the HSC requires that the expectations of the input and output (E_{in} and E_{out}) satisfy a function, that is, $E_{out} = f(E_{in})$. However, the length of the HSC is limited. Thus, it uses the mean values of the HSN to replace the expectation, that is, the input and output (M_{in} and M_{out}) is $M_{out} = f(M_{in})$. $S(t)$ represents the HSN number S , at time t , and (16) shows the relationship between $S(t)$, the mean value (M), and the summation (Sum). L is the stream length of $S(t)$, $t = 1, 2, \dots, L$

$$\text{Sum} = \sum_{t=1}^L S(t) = M \cdot L. \quad (16)$$

According to (16), the functional relationship between M_{in} and M_{out} can be transformed as a function of Sum_{in} and Sum_{out} , as shown in the following equation:

$$M_{out} = \frac{\text{Sum}_{out}}{L} = f\left(\frac{\text{Sum}_{in}}{L}\right) = f(M_{in}). \quad (17)$$

Thus, the relationship between Sum_{in} and Sum_{out} is obtained. The function $g(x)$ is the stretching transformation of $f(x)$, as follows:

$$\text{Sum}_{out} = g(\text{Sum}_{in}) = L \cdot f\left(\frac{\text{Sum}_{in}}{L}\right). \quad (18)$$

When L is close to infinity, (17) is changed as follows:

$$\begin{aligned} E_{out} &= \lim_{L \rightarrow \infty} (M_{out}) = \lim_{L \rightarrow \infty} \frac{g(\text{Sum}_{in})}{L} \\ &= f\left(\lim_{L \rightarrow \infty} \frac{\text{Sum}_{in}}{L}\right) = f(E_{in}). \end{aligned} \quad (19)$$

To achieve the above relationship, $\text{Sum}_{out} = g(\text{Sum}_{in})$, and the output HSN be obtained according to (20), which is the

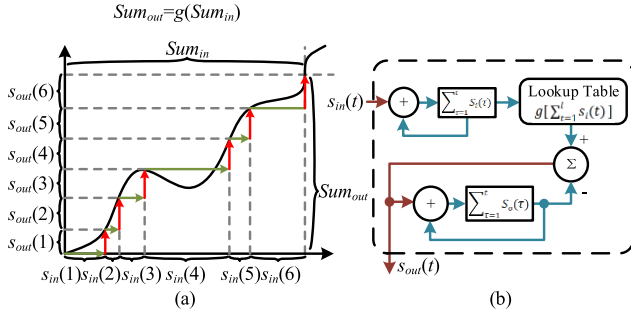


Fig. 7. Function calculation realized using successive approximation method. (a) Example for the process of successive approximation. (b) Circuit of successive approximation method.

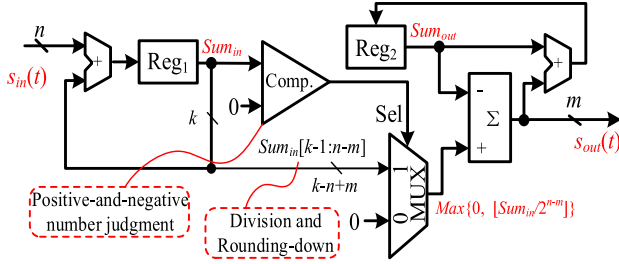


Fig. 8. $Relu()$ function circuit realized using successive approximation method.

process of successive approximations, as shown in Fig. 7(a)

$$S_{out}(t) = \begin{cases} g[S_{in}(1)], & t = 1 \\ g\left[\sum_{\tau=1}^t S_{in}(\tau)\right] - \sum_{\tau=1}^{t-1} S_{out}(\tau), & t > 1. \end{cases} \quad (20)$$

The sum of the output HSC is then obtained using the following equation, which realizes the function calculation:

$$\begin{aligned} Sum_{out} &= \sum_{\tau=1}^L S_{out}(\tau) \\ &= g\left[\sum_{\tau=1}^L S_{in}(\tau)\right] = g(Sum_{in}). \end{aligned} \quad (21)$$

The circuit is shown in Fig. 7(b). $S_i(t)$ is received by the accumulator to obtain $\sum_{\tau=1}^t S_{in}(\tau)$. The function calculation, $g(x)$, can be realized using a lookup table (LUT) that can fit any function. $g(x)$ is obtained from the origin function $f(x)$ according to (18). When the output of $Relu()$ is $S_{out}(t)$, the Reg2 accumulator receives $S_{out}(t)$ to simultaneously obtain $\sum_{\tau=1}^t S_{out}(\tau)$.

$Relu()$ function is a common activation function for the neural networks. It is implemented by using the successive approximation method. The LUT shown in Fig. 7(b) can be replaced by the positive-and-negative number judgment circuits in Fig. 8, which reduces the hardware cost. $LeakyRelu()$ can also be implemented without LUT. The difference is that the input of port0 of MUX is $a \sum_{\tau=1}^t S_{in}(\tau)$ instead of 0 and a is a constant coefficient for $LeakyRelu()$ to simplify the circuit, a is recommended to be 2^{-k} , $k \in \mathbb{Z}$, and multiplication can be achieved using a shift operation.

In addition, the output of HSN can be truncated to reduce the bit width. Considering $Relu()$ as an example in Fig. 8, the

input bitwidth of HSC is n -bit. To obtain the m -bit output HSN, the activation functions of (20) and (21) are replaced by the following equations:

$$\begin{aligned} Sum_{out} &= \sum_{\tau=1}^L S_{out}(\tau) \\ &= \left\lfloor \frac{g\left[\sum_{\tau=1}^L S_{in}(\tau)\right]}{2^{n-m}} \right\rfloor = \left\lfloor \frac{g(Sum_{in})}{2^{n-m}} \right\rfloor \quad (22) \\ S_{out}(t) &= \begin{cases} \left\lfloor \frac{g[S_{in}(1)]}{2^{n-m}} \right\rfloor, & t = 1 \\ \left\lfloor \frac{g\left[\sum_{\tau=1}^t S_{in}(\tau)\right]}{2^{n-m}} \right\rfloor - \sum_{\tau=1}^{t-1} S_{out}(\tau), & t > 1 \end{cases} \quad (23) \end{aligned}$$

where $\lfloor \cdot \rfloor$ represents the rounding-down operation. In general, the truncation should take place subsequent to the $Relu()$ operation as in (22). However, it occurs prior to the $Relu()$ operation in Fig. 8. This adjustment is to reduce the input bit width of MUX and has no influence on the computing results, which is a special design for $Relu()$ or $LeakyRelu()$. When calculating other complex functions, the complex functions can also truncate the output HSN based on (22) and (23). This scheme can flexibly adjust the bit width of the output HSN, which helps to reduce the hardware overhead of the next cascade circuit.

B. Neuron Circuit

In a conventional SC, data must be repeatedly converted between the SN and BN to avoid the impact of independence. Consequently, it is difficult for the circuit to realize a pipeline architecture, and a large amount of processing time is required for data conversion, which leads to high latency. Thus, from the computing efficiency, that is operations per joule (or per area) per second, due to the huge disadvantage in information efficiency and encoding cost, the energy (or area) efficiency of SC in reality is usually worse than that of traditional binary computing. The low-cost advantages of SC have not yet been fully exploited. This problem is solved in the proposed HSC, where the data no longer needs to be converted between different formats, thus saving time cost and hardware overhead of conversion. The HSC method can demonstrate its advantages in large parallel and complex computational situations, such as the neural networks.

The neuron circuit was designed based on HSC, as shown in Fig. 9. A binary adder is used as an HSC adder in the design. Although binary adder costs a larger area than MUX adder, it can achieve higher computing accuracy and lower latency. $S_1(t), S_2(t), \dots, S_i(t)$ are n -bit HSNs representing the inputs of neuron, $w_1, w_2, \dots, w_j$, and bias are the parameters of the neurons that are stored as BN. The neuron uses $Relu()$ as an activation function, as shown in Fig. 8. $Relu()$ also performs a truncation to ensure that the input and output HSN have the same bit width. The same bit widths of the input and output HSNs were used to realize the cascade connection of multiple neurons. Thus, the pipeline architecture of a neural

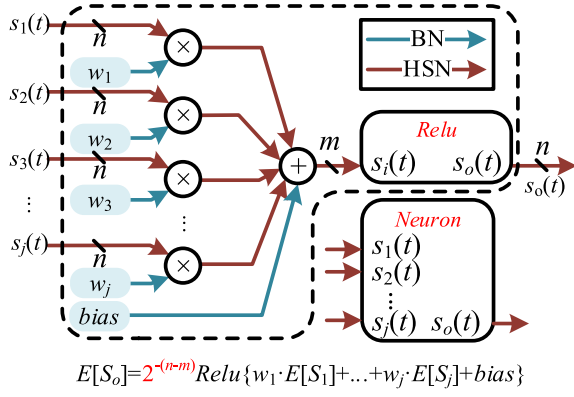


Fig. 9. Neuron structure based on HSC without any converters of weight and bias data. The input and output are HSN, and the bit width of output HSN can be adjusted.

TABLE I

HARDWARE RESULT FOR NEURON CIRCUIT WITH DIFFERENT NUMBER OF SYNAPSES

Number of Synapses	Method		LUT	FF	Power (W)	Power per Synapse (mW)	L
32	HSC	1-bit	388	344	0.02	0.63	256
		2-bit	631	453	0.031	0.97	128
	B. (8-bit)		3142	1065	0.105	3.28	1
64	HSC	1-bit	736	667	0.036	0.56	256
		2-bit	951	786	0.046	0.72	128
	B. (8-bit)		6249	2151	0.21	3.28	1

All data were obtained from the companion design suite of FPGA, KCU116. Clock frequency is 300 MHz. Bit width of input HSC is 1-bit or 2-bit. The weight and bias are 8-bit BN. The circuits of binary are automatically optimized from the design suite. B. represent Binary, L. represents Latency.

network can be formed to achieve low-latency performance. In addition, the data conversion problem of conventional SC, which is the repeated conversion between SN and BN, was also addressed.

Table I shows the implementation results of the neuron circuit for different number of synapses based on HSC and traditional binary, where the circuit of the HSC is implemented according to Fig. 9. $S_i(t)$ is set to 1-bit HSN, w_j , and bias is set to 8-bit signed BN. As the number of synapses increases, the cost of the $\text{Relu}()$ circuit is averaged across each synapse. Thus, the average power consumption in the case of 64 inputs is slightly lower than that of 32 inputs. The hardware requirements of neurons realized by HSC (1-bit HSN) were less than 20% of that of neurons realized by binary. The advantage of hardware cost for HSC can be further expanded with the increase of bit width. The network parameters are stored directly near the computing unit without repeatedly reading data from the external memory. Thus, it avoids the problem that the speed of artificial intelligence (AI) accelerator with binary is constrained by the speed of scheduling data.

C. Application of Neural Network

Based on the neuron circuit shown in Fig. 9, a five-layer fully connected neural network is constructed to classify

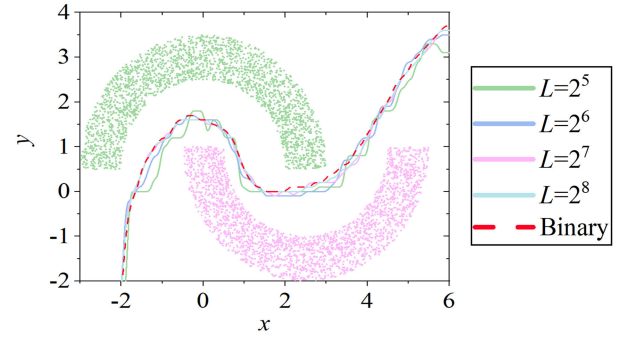


Fig. 10. Classification of two linear inseparability moons using deep network; L is the stream length.

TABLE II

FPGA-BASED HARDWARE IMPLEMENTATION OF DEEP NEURAL NETWORK

Bit Width of Quantization	LUT	FF	Power (W)
8-bit	53110	47503	2.389
Number of synapses	Clock Frequency (MHz)	computation Performance (TOPS)	Energy Efficiency (TOPS/W)
4544	300	2.726/L	1.141/L

All data were obtained from the companion design suite of FPGA, KCU116. L represents the bitstream length of HSC.

two linearly inseparability moons. The structure of the deep network was [2, 64, 32, 32, 32, 2]. The classification results are shown in Fig. 10. Deep network training was performed using software (PyTorch and MATLAB). The weight and the bias of the network, including the inputs x and y , were subjected to 8-bit quantization. The boundary of the binary classification is indicated by the red dashed line in Fig. 10. The classification boundaries based on HSC with different stream lengths are drawn as solid lines with different colors. When $L \geq 2^6$, the difference in the boundary between the HSC and the binary is still very small. Even if L decreased to 2^5 , the boundary obtained from the HSC could clearly distinguish between the two sets of points.

This neural network with HSC was verified using a hardware circuit. The main feature of the network with HSC is that it is the nonindependence of input data instead of the independence requirement of traditional SC when multiple multiplications. The network in the test contained 4544 synapses, which consisted of 4544 multiplies-accumulation (MAC). The implementation results are shown in Table II. The average hardware size of multiplier and adder is only 11.7 LUTs and 10.5 FFs. Under a 300-MHz clock, it can perform 2.726×10^{12} HSC operations per second (OPS) with a power of 2.389 W, equivalent to $2.726/L \times 10^{12}$ binary OPS. L is the bitstream length, which depends on accuracy requirements.

The full connect (FC) neural network above based on HSC was implemented and fabricated by 40-nm CMOS technology. The structure of the deep neural network was set as [64, 32, 32, 32, 10] for classification on MNIST. Network training was performed using PyTorch. Because the

TABLE III
COMPARISON WITH OTHER NEURAL NETWORK DESIGNS

Method		SC				BN				Proposed HSC (8-BIT)			
		Conventional			Pipeline	Analog	16-bit		8-bit				
		[30]	[31]	[32]	[33]	[34]	[35]	[36]	[37]				[38]
Process		ASIC 45nm	ASIC 45nm	ASIC 45nm	ASIC 45nm	ASIC 65nm	FPGA VC709	FPGA GX1150	FPGA VX690T	ASIC 16nm	FPGA KCU116		ASIC 40nm
Number of MAC		25	256	512	550	-	2144	1320	1027	1024	10332	6804	4544
Power (mW)		1.28	1490	2619.8	651	33.17	30200	37448	9180	4160	5852	3985	102.3
Clock (MHz)		1136	561	1563	200	-	156	370	200	2001	300		400
Area	mm ² for ASIC)	0.00085	0.0012	1.09	2.01	1.321	-	-	-	3.1	-	-	0.53
	LUT for FPGA	-	-	-	-	-	273805	437000	231761	-	115143	78401	-
Length of SN (<i>L</i>)		2048	1024	1024	512	8	-	-	-	-	32		
Latency (μs)		1.8	1.83	0.66	2.56	-	-	-	-	-	0.11		0.08
Network Model		Lenet5 Conv×3 FC×3					AlexNet Conv×5 FC×3	-	VGG16 Conv×13 FC×3	-	Conv×8	Conv×4	FC×4
Data Set		MNIST		Cifar10		MNIST	ImageNet	-	ImageNet	-	Cifar10	MNIST	MNIST
Top-1 accuracy (%)		99.12	99.07	88	91	99.06	62.5	-	63.74	-	86.99	99.1	96.01
Operation per Second (GOPS)		0.028	0.28	1.563	220	-	565.94	1790	760.83	4098	193.7	127.56	113.6
Energy Efficiency (GOPS/W)		22	0.18	0.596	340	1039	22.15	47.8	82.88	985.1	33.10	32.01	1109.8
Length×Area per MAC (only ASIC)		0.07	0.0048	2.18	1.87	-	-	-	-	-	-	-	0.0037
Length×Power per MAC per MHz		0.092	10.62	3.35	3.03	-	-	-	-	-	0.0604	0.0625	0.0018

Data were obtained from the companion design suite of FPGA KCU116 and 40 nm low-power CMOS process.

photos were resized as 8×8 from 28×28 to decrease the number of MACs, the misclassification rate increased to 3.89% using floating point number in software. The weights and biases of the inputs data of the network, were quantized to 8-bit INT by MATLAB based on range linear symmetric quantization method with a misclassification rate of 3.96%.

A die micrograph of the neural network is shown in Fig. 11, where the die area is 1.0 mm^2 , and the core area of the neural network is only $0.73 \times 0.73 \text{ mm}^2$. The network structure consists of 4544 number of multiplies-accumulation (MAC), and the implementation results by application-specified integrated circuit (ASIC) are shown in Table III. Several designs based on SC or binary are listed in Table III. Conventional SC designs [30], [31], [32] are limited by the disadvantage of a long bitstream length, and their energy efficiency is lower than that of binary designs implemented by ASIC. The design [33] using a pipeline architecture exhibited high energy efficiency. The design [34] using an analog circuit to realize SC significantly improved the energy efficiency; however, only the first layer of their network was realized using SC. When the HSC network implemented by ASIC, it can perform 3.635×10^{12} HSC OPS ($3.635 \times 10^{12}/L = 1.136 \times 10^{11}$ binary OPS) with a power of 102.3 mW under a 400-MHz clock. One SOPS = $1/L$ OPS, where L is the bitstream length, which depends on the accuracy requirements. The area efficiency was 6.82 TSOPS/mm^2 , and the energy efficiency was $142.6\text{--}35.53 \text{ TSOPS/W}$ (here, $L = 8\text{--}32$). The HSC neural network ($L = 32$) achieved more than 50 times the energy efficiency of conventional SC designs, which is

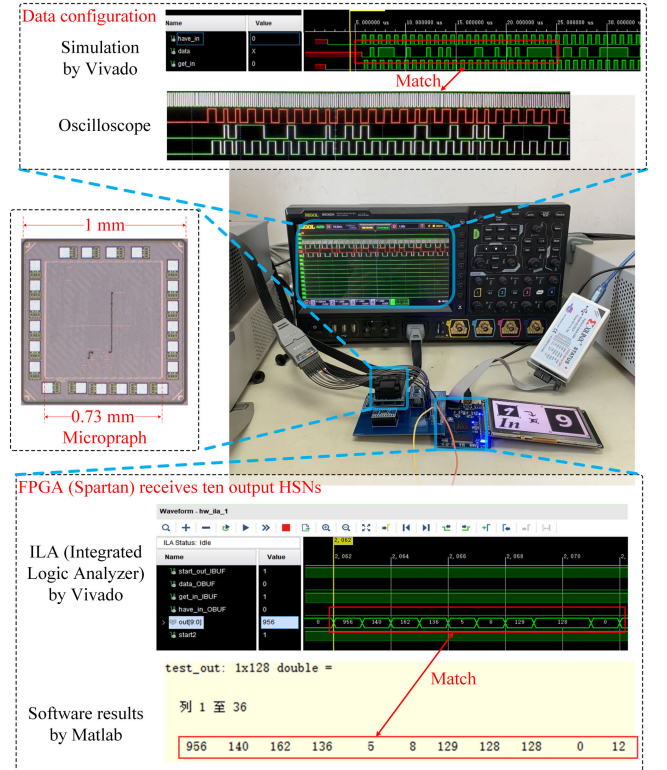


Fig. 11. Chip micrograph and numeral recognition experiment of neural network based on HSC. Accuracy reaches 96.01% on MNIST when $L = 32$. The photograph is resized as 8×8 for the reduction in number of MACs.

similar to the performance of analog or binary-number neural networks. Compared to an 8-bit int neural network, a network

based on HSC has very little loss in Top-1 accuracy, which has 0.03% decrease, from 96.01% to 96.04%. Benefits from the robustness of the neural network, small sacrifice on computational accuracy have little impact on the inference accuracy. And, it is also verified by previous works [30], [31], [32], [33], [34], these SC designs all achieve good inference accuracy. When the bitstream length L is compressed to eight, the energy efficiency of the neural network can be four times than $L = 32$ although the miss rates will be increases to 11.3%.

At the same time, we also implemented two kinds of CNN models, which are four-layer CNN $[(1, 8, 3), (8, 8, 3) \times 2, \text{ and } (8, 10, 3)]$ and eight-layer CNN $[(3, 32, 3), (32, 32, 3) \times 6, (32, 10, 3)]$, (i, o, k) means (in channels, out channels, and kernel size). Two kinds of CNN were both implemented using FPGA KCU116, and tested on the datasets MNIST and Cifar-10, respectively. The experimental results are shown in Table III. The CNN based on FPGA can perform 127.56 and 193.7 GOPS with number of MAC 6804 and 10332 under a 300-MHz clock.

IV. CONCLUSION

In this article, we proposed a new number representation called HSN and HSC method. This study first propounded the number representation from BN and SC to HSN, unifying the number representation and demonstrating the fundamental properties of the HSC. The HSN is represented by the multibit stream stochastic number depending on the weight of position number system. The representations of 1-bit stream SN and BNs are two extremes and two types of special examples of HSN. The method of the expectation of multibit streams proposed can achieve the complex computation of HSC. Compared with the conventional SC, HSC realized high energy efficiency, lower latency, and completely accurate computation. The rationality of the computing rules for multibit streams is demonstrated and some basic arithmetic circuits for the HSC system are designed too.

The circuits of activation function, neuro, and deep neural network with HSC were designed in this article. Five-layer neural network with an HSC was realized for a handwritten numeral recognition experiment of a 40-nm ASIC. The HSC network achieved more than 50 $\times$ the energy efficiency of conventional SC designs, which is similar to the performance of analog or binary designs. These applications illustrated that the HSC computing method has high performance than the traditional SC, which avoids the obstacles of the conversion and latency in traditional SC. Thus, it demonstrates the main advantages of HSN, HSC theory to edge computing systems and shows good application prospects in massively parallel computing.

REFERENCES

- [1] A. Avizienis, "Signed-digit number representations for fast parallel arithmetic," *IEEE Trans. Electron. Comput.*, vol. EC-10, no. 3, pp. 389–400, Sep. 1961.
- [2] B. Parhami, "Generalized signed-digit number systems: A unifying framework for redundant number representations," *IEEE Trans. Comput.*, vol. 39, no. 1, pp. 89–98, Jan. 1990.
- [3] B. R. Gaines, "Stochastic computing," in *Proc. Spring Joint Comput. Conf. AFIPS (Spring)*, 1967, pp. 149–156, doi: [10.1145/1465482.1465505](https://doi.org/10.1145/1465482.1465505).
- [4] W. J. Poppelbaum, C. Afuso, and J. W. Esch, "Stochastic computing elements and systems," in *Proc. Fall Joint Comput. Conf. AFIPS (Fall)*, 1967, pp. 635–644.
- [5] B. D. Brown and H. C. Card, "Stochastic neural computation. I. Computational elements," *IEEE Trans. Comput.*, vol. 50, no. 9, pp. 891–905, Sep. 2001.
- [6] A. Alaghi and J. P. Hayes, "Survey of stochastic computing," *ACM Trans. Embedded Comput. Syst.*, vol. 12, no. 2s, pp. 1–19, May 2013.
- [7] J. P. Hayes, "Introduction to stochastic computing and its challenges," in *Proc. DAC*, no. 59, 2015, pp. 1–3.
- [8] B. R. Gaines, "Stochastic computing systems," in *Advances in Information Systems Science*. Boston, MA, USA: Springer-Verlag, 1969, pp. 37–172.
- [9] A. Alaghi, W. Qian, and J. P. Hayes, "The promise and challenge of stochastic computing," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 37, no. 8, pp. 1515–1531, Aug. 2018.
- [10] S. R. Faraji and K. Bazargan, "Hybrid binary-unary hardware accelerator," *IEEE Trans. Comput.*, vol. 69, no. 9, pp. 1308–1319, Sep. 2020.
- [11] L. Sousa, "Nonconventional computer arithmetic circuits, systems and applications," *IEEE Circuits Syst. Mag.*, vol. 21, no. 1, pp. 6–40, 1st Quart., 2021.
- [12] A. Khataei, G. Singh, and K. Bazargan, "Approximate hybrid binary-unary computing with applications in BERT language model and image processing," in *Proc. ACM/SIGDA Int. Symp. Field Program. Gate Arrays*, Feb. 2023, pp. 165–175.
- [13] S. Liu and J. Han, "Energy efficient stochastic computing with sobol sequences," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2017, pp. 650–653.
- [14] S. Aygun, L. Kouhalvandi, M. H. Najafi, S. Ozoguz, and E. O. Gunes, "Hardware-software co-optimization of long-latency stochastic computing," *IEEE Embedded Syst. Lett.*, early access, Sep. 25, 2023, doi: [10.1109/LES.2023.3298734](https://doi.org/10.1109/LES.2023.3298734).
- [15] X. Tang et al., "Delta sigma modulator-based dividers for accurate and low latency stochastic computing systems," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 13, no. 1, pp. 270–284, Mar. 2023.
- [16] M. H. Najafi, D. Jenson, D. J. Lilja, and M. D. Riedel, "Performing stochastic computation deterministically," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 12, pp. 2925–2938, Dec. 2019.
- [17] J. Wang, H. Chen, D. Wang, K. Mei, S. Zhang, and X. Fan, "A noise-driven heterogeneous stochastic computing multiplier for heuristic precision improvement in energy-efficient DNNs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 2, pp. 630–643, Feb. 2023.
- [18] Z. Xia, J. Chen, Q. Huang, J. Luo, and J. Hu, "Neural synaptic plasticity-inspired computing: A high computing efficient deep convolutional neural network accelerator," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 2, pp. 728–740, Feb. 2021.
- [19] H. Chen and J. Han, "Stochastic computational models for accurate reliability evaluation of logic circuits," in *Proc. 20th Symp. Great lakes Symp. (VLSI)*, May 2010, pp. 61–66.
- [20] H. Aliee and H. R. Zarandi, "Fault tree analysis using stochastic logic: A reliable and high speed computing," in *Proc. Annu. Rel. Maintainability Symp.*, Jan. 2011, pp. 1–6.
- [21] H. Sim, D. Nguyen, J. Lee, and K. Choi, "Scalable stochastic-computing accelerator for convolutional neural networks," in *Proc. 22nd Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jan. 2017, pp. 696–701.
- [22] N. Temenos and P. P. Sotiriadis, "A stochastic computing sigma-delta adder architecture for efficient neural network design," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 13, no. 1, pp. 285–294, Mar. 2023.
- [23] S. Khoram, K. Daruwalla, and M. Lipasti, "Energy-efficient Bayesian inference using bitstream computing," *IEEE Comput. Archit. Lett.*, vol. 22, no. 1, pp. 37–40, Jan. 2023.
- [24] P. Li, D. J. Lilja, W. Qian, K. Bazargan, and M. D. Riedel, "Computation on stochastic bit streams digital image processing case studies," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 22, no. 3, pp. 449–462, Mar. 2014.
- [25] W. J. Poppelbaum, "Statistical processors," *Adv. Comput.*, vol. 14, pp. 187–230, Jan. 1976.
- [26] K. K. Parhi, "Analysis of stochastic logic circuits in unipolar, bipolar and hybrid formats," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2017, pp. 1–4.
- [27] V. Canals, A. Morro, A. Oliver, M. L. Alomar, and J. L. Rosselló, "A new stochastic computing methodology for efficient neural network implementation," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 27, no. 3, pp. 551–564, Mar. 2016.

- [28] Y. Chen and H. Li, "Stochastic computing using amplitude and frequency encoding," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 5, pp. 656–660, May 2022.
- [29] H. Li and Y. Chen, "Hybrid logic computing of binary and stochastic," *IEEE Embedded Syst. Lett.*, vol. 14, no. 4, pp. 171–174, Dec. 2022.
- [30] S. R. Faraji, M. H. Najafi, B. Li, D. J. Lilja, and K. Bazargan, "Energy-efficient convolutional neural networks with deterministic bit-stream processing," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2019, pp. 1757–1762.
- [31] Z. Li et al., "HEIF: Highly efficient stochastic computing-based inference framework for deep neural networks," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 38, no. 8, pp. 1543–1556, Aug. 2019.
- [32] A. Zhakatayev, S. Lee, H. Sim, and J. Lee, "Sign-magnitude SC: Getting 10X accuracy for free in stochastic computing for deep neural networks," in *Proc. 55th ACM/ESDA/IEEE Design Autom. Conf. (DAC)*, Jun. 2018, pp. 1–6, doi: [10.1109/DAC.2018.8465807](https://doi.org/10.1109/DAC.2018.8465807).
- [33] C. F. Frasser et al., "Fully parallel stochastic computing hardware implementation of convolutional neural networks for edge computing applications," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Apr. 22, 2022, doi: [10.1109/TNNLS.2022.3166799](https://doi.org/10.1109/TNNLS.2022.3166799).
- [34] V. T. Lee, A. Alaghi, J. P. Hayes, V. Sathe, and L. Ceze, "Energy-efficient hybrid stochastic-binary neural networks for near-sensor computing," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2017, pp. 13–18, doi: [10.23919/DATE.2017.7926951](https://doi.org/10.23919/DATE.2017.7926951).
- [35] H. X. Fan, L. Jiao, W. Cao, X. Zhou, and L. Wang, "A high performance FPGA-based accelerator for large-scale convolutional neural networks," in *Proc. 26th Int. Conf. Field Program. Logic Appl. (FPL)*, Sep. 2016, pp. 1–9.
- [36] J. Zhang and J. Li, "Improving the performance of OpenCL-based FPGA accelerator for convolutional neural network," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays*, Feb. 2017, pp. 25–34.
- [37] X. Lian, Z. Liu, Z. Song, J. Dai, W. Zhou, and X. Ji, "High-performance FPGA-based CNN accelerator with block-floating-point arithmetic," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 8, pp. 1874–1885, Aug. 2019.
- [38] B. Zimmer et al., "A 0.32–128 TOPS, scalable multi-chip-module-based deep neural network inference accelerator with ground-referenced signaling in 16 nm," *IEEE J. Solid-State Circuits*, vol. 55, no. 4, pp. 920–932, Apr. 2020.



Hongge Li (Member, IEEE) received the Ph.D. degree in engineering from the Graduate School of Information Sciences, Tohoku University, Sendai, Japan, in 2005.

From 2006 to 2008, he was an Assistant Professor with the Department of Bioengineering and Robotics, Tohoku University. He is currently a Professor with the School of Electronic Information Engineering, Beihang University, Beijing, China. He has authored or coauthored more than 90 scientific articles in journals and international conferences, including the IEEE JOURNAL OF SOLID-STATE CIRCUITS (JSSC), IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI), TCASII, IEEE EMBEDDED SYSTEMS LETTERS (ESL), IEEE/OSA JOURNAL OF DISPLAY TECHNOLOGY (JDT), IEEE TRANSACTIONS ON ELECTROMAGNETIC COMPATIBILITY (TEM), *Neurocomputing*, *Sensors*, *Microelectronics Journal*, *Microelectronics Reliability*, *Electronics Letters*, *JSID*, and IEEE International Symposium on Circuits and Systems (ISCAS), and is an inventor of 20 patents. His current research interests include Stochastic computing (SC) circuits, panel display drivers/touch chip, neural network, electromagnetic interferences on chip, signal processing, and crossover study methodologies for real-time hybrid systems.



Yuhao Chen (Student Member, IEEE) received the B.S. degree in electrical engineering from Beihang University, Beijing, China, in 2019, where he is currently working toward the Ph.D. degree.

His current research interests include low-power design, stochastic computing, neural networks, and neuromorphic computing.